REQUI TRADING · ENGINEERING

Trailing & Exit Engine Development Guide

Version 2.4 — automatic trailing stops, T1/T2 scale-outs, immediate-trail stop mode, manual overrides, broker-side reconciliation, disaster stop, and the live trade stream

---

Document type: Engineering handoff — full source, diffs, and rollout steps

Companion to: Autonomous Execution Development Guide v2.2 (auto-execute, orders, P&L)

Date: August 5, 2026 · Classification: Confidential — internal engineering only

Contents

1 What this version ships

Versions 2.3 and 2.4 (delivered together here as the **exit engine**) turn the Autonomous module into a fully self-managing trading system: every position the system opens is protected, scaled out of, and exited automatically, with the human able to override at any moment. Six capabilities ship on top of the v2.2 base (auto-execute, order history, P&L, strategy self-identification):

1. **Automatic trailing stops.** A 5-second-cycle module manages every open position that carries a governed stop — whether the entry came from the autonomous engine or from an Intelligence chat trade, and whether it was human-confirmed or auto-executed. Flat stop first, then a ratcheting ATR trail that never moves down, with breakeven and T1 floors after scale-outs.
2. **Immediate-trail stop mode.** A per-user alternative stop policy: no flat stop-loss anywhere — a percentage trail is live from entry at 0.5%, tightening on gain milestones, with a moving hard bottom. Selected from the UI, persisted per user.
3. **Automatic scale-outs at T1/T2.** The module sells a third of the position at the first target and half the remainder at the second, booking realized P&L and advancing the trailing floors. The trail runs the rest.
4. **Manual override controls.** Per-position tighten (stop or trail), take/return manual control (the only way to loosen), and flatten-now — all from the positions table, all audited.
5. **Broker-side reconciliation.** Cancel-and-govern: resting broker-side sell-stops the system did not create are cancelled with a confirmation alert, so they can never preempt the trail. Disaster stop: one catastrophic resting STP far below everything as crash insurance, polled, and auto-cancelled on close.
6. **Live trade stream.** A real-time activity feed on the Autonomous page — entries, scale-outs, fires, cancellations, overrides — updating every five seconds.

> **READ THIS FIRST**
>
> The exit engine **exits only** — it can never open or add to a position. Every exit travels the same broker adapter path as entries, with the same idempotency discipline and the same dead-end-on-UNKNOWN rule. All stops are managed in software; the only resting broker order the system ever places is the optional disaster stop, which sits far below the working stop and is removed when the position closes.

2 The trailing-stop spec and how it maps to code

The module was built to a five-point specification. The mapping below is the reference for every later section:

Table 1 — Spec → implementation

Spec	Requirement	Implementation
1. ARM	Trail activates only after price reaches entry + 1R (R = entry − original stop). Before that, the original flat stop governs.	`evaluate()` CLASSIC branch: `armed` flips when `last ≥ entry + 1×R`; pre-arm the fire level is the flat stop itself, so a position is never unprotected.
2. TRAIL	Trail = highest price since entry − (2.5 × ATR14). Recalculate on every new high; NEVER move down.	`highestPrice` ratchets in the DB; `trail = max(stored trail, highest − 2.5×ATR14, floor)`. ATR is the same Wilder ATR the signal engine uses (`api/engine/triggers.ts`).
3. FLOOR	After T1 scale-out fills, trail can never be below entry (breakeven floor). After T2, floor moves to T1 price.	`scaleStage` on the position row (0/1/2) drives the floor: stage ≥ 1 → entry, stage ≥ 2 → T1 price.
4. FIRE	If last price ≤ trail → market sell the entire remaining position. Log: entry, trail at fire, fill, slippage.	`submitExit()` sends a pure MKT sell through the position's own broker (paper / Robinhood MCP / IBKR, resolved from the source ticket). `bookExit()` writes the alert with entry, reference at fire, fill, and slippage.
5. SAFETY	Feed stalls >60s → flatten position or alert human. Kill switch checked every cycle.	Stall = no usable mark for one full refresh cycle (75s grace): `safetyFlatten()` attempts the flatten AND raises a CRITICAL alert (throttled 5 min). The per-user kill switch (`ai_limits.killSwitch`, persisted in the DB) is read every cycle — ON means the bot stands down entirely.

Two structural properties worth knowing: the module manages positions, not tickets — so it covers engine trades and chat-brokered trades identically. And it sleeps outside US regular hours (09:30–16:00 ET, Mon–Fri): overnight, positions rest on their stops and the disaster order.

3 Stop modes — CLASSIC vs IMMEDIATE_TRAIL

Version 2.4 adds a per-user stop policy (`users.stopMode`, default CLASSIC), switched from the Trail toggle in the Orders & P&L panel header. Both modes share the same machinery — feed handling, ratchets, scale-outs, fire path, alerts — and differ only in how the fire level is computed.

3.1 CLASSIC (default)

The Section 2 behaviour: flat stop until entry + 1R, then the 2.5×ATR14 trail with scale-stage floors. Best for swing-style entries that need room to develop.

3.2 IMMEDIATE_TRAIL

No flat stop-loss is used anywhere — a software trail is live from the first second, starting at the same margin a flat stop would have used (0.5% below price), and tightening as the trade proves itself:

Table 2 — IMMEDIATE_TRAIL distance schedule (implemented in immediateDistPct)

Gain since entry	Trail distance below highest	Hard bottom
0% (at entry)	0.50%	entry − 0.5%
+0.5%	0.45%	entry − 0.5%
+1.0%	0.40%	**entry (breakeven)**
+1.5%	0.32%	entry
+2.0%	0.27%	entry + 40% of peak gain
+2.5% and beyond	0.22% (floor — never tighter)	entry + 40% of peak gain, ratcheting

Two calibration rules protect the schedule from noise: the distance never goes below **0.5 × ATR14** (jumpy names get breathing room even when the percentage says tighter), and the percentage itself never goes below 0.22% (a single bar's wiggle cannot kill a runner). Fire = `last ≤ max(trail, hard bottom)`. The hard bottom is a pure function of entry and the highest price — it needs no storage and can never move down.

> **NO BROKER-SIDE STOP-LOSS**
> In IMMEDIATE_TRAIL mode there is deliberately no broker-side stop-loss at all — the software trail and hard bottom govern everything, which removes any confusion about two competing stop authorities. The disaster stop (Section 6) still rests far below as crash insurance on live brokers.

4 Automatic scale-outs at T1 and T2

The spec's floors assumed T1/T2 scale-outs would fill — but until v2.4 nothing placed them. The exit engine closes that gap:

- **Trigger.** After the fire check (protection always wins), if `last ≥ T1` at stage 0 → sell `round(qty/3)` at market. If `last ≥ T2` at stage 1 → sell `floor(remaining/2)`. Positions under 3 shares skip scaling — the trail runs everything.
- **Plumbing.** Engine proposals carry both targets: `monitor.ts` passes `proposal.targets[1]` into the ticket's new `target2` column, and `recordFill` seeds `positions.t2Price` from it. T1 already flowed the same way.
- **Booking.** Fills go through the same submission/poll path as protective exits; `bookScaleOut()` splits the position row (realized P&L on the closed tranche, stage advanced on the remainder), and the alert records entry, target, fill, realized P&L, and the new floor ("BREAKEVEN" after T1, "T1 PRICE" after T2).
- **Manual sells count too.** A partial sell via a confirmed ticket also advances the stage — the floor rules activate no matter who scaled.

5 Manual override controls

Every open position row in the Orders & P&L panel carries three actions. The governing invariant: **you can always tighten protection instantly, but loosening requires explicitly taking the position out of the module's hands** — a moment of emotion can never silently strip a position of protection.

Table 3 — Override actions (all user-scoped, all write an audit alert)

Action	Behaviour	Validation
TIGHTEN	Raise the flat stop (pre-arm) or the trail (armed) toward the current price. Setting a trail on an unarmed position arms it at your level; the module ratchets upward from there.	New level must be above the current one (the ratchet never moves down) and below the live mark — at/above the mark suggests Flatten instead.
MANUAL / AUTO	Take full ownership of the exit: the module stands down for that position — no trail updates, no fires, no scale-outs, no safety flatten — and a red MANUAL badge shows on the row. Press AUTO to hand it back.	Per-position toggle (`positions.overrideMode`). The resting disaster stop stays in place as insurance even under manual control.
FLATTEN	Market-sell the entire position now, through the same audited fire path as any module exit (same broker, same polling, same fill/slippage logging).	Browser confirm first. Works any time, including off-hours — `requestFlatten` calls the broker directly rather than waiting for the loop.

6 Broker-side reconciliation — cancel-and-govern and the disaster stop

Once per position (its first management cycle), the module reconciles the broker's world with ours.

6.1 Cancel-and-govern

A stop-loss placed outside our system — keyed into TWS by hand, left by another tool — can fire before the trail and sell the position out from under the ledger, leaving our books showing a position that no longer exists. To prevent exactly one exit authority per position:

1. The broker adapter contract gains an optional `getOpenOrders(accountId)` capability, implemented for IBKR (Client Portal `/iserver/account/orders`, live statuses only).
2. The module lists open orders and cancels every resting SELL stop on the position's symbol that it did not create — anything not tagged with our `TRAIL-`/`DSTR-` client-order-id prefixes and not our own disaster order.
3. Each cancellation posts a confirmation alert naming the order and its level ("Broker-side stop canceled — trailing module governs this exit"). A failed cancel escalates to CRITICAL with instructions to cancel manually. Brokers without an open-orders endpoint get a one-time "verify manually" alert instead — nothing is silently skipped.

6.2 Disaster stop

Pure crash insurance, deliberately non-overlapping with the working stops:

- **Placement.** One resting STP sell, GTC, at `flat stop − max(1% of entry, ATR14)` — far below everything the module manages. Tagged `DSTR-{positionId}`, its order id and price stored on the position row. Skipped on the paper adapter (the simulator fills resting orders instantly, which would book a fake loss).
- **Supervision.** Polled once a minute. If it ever FILLED, the broker closed the position — the books close honestly as `DISASTER_STOP` with a CRITICAL alert telling you to investigate the feed/module. If it vanishes (cancelled/rejected), the module re-places it.
- **Cleanup.** Cancelled automatically the moment the position closes by any other path — no orphan resting orders are ever left behind. Rows carrying one show a +D badge in the UI.

7 Live trade stream

The Autonomous page gains a **Live trade stream** panel — the "watch the bot buying and selling" view. It polls `engine.activity` every 5 seconds and renders the execution-relevant alert feed (entries, scale-outs, trail fires, cancellations, overrides, safety events) newest-first with ET timestamps, severity colour coding (teal fills, sky scale-outs, amber stops, red criticals), and a pulsing live indicator. No new storage — it projects the alert feed that the exit engine and ticket layer already write, so the stream can never disagree with the audit trail. Source: `streamActivity()` in portfolio.ts (Appendix B) and `src/components/LiveStream.tsx` (Appendix J).

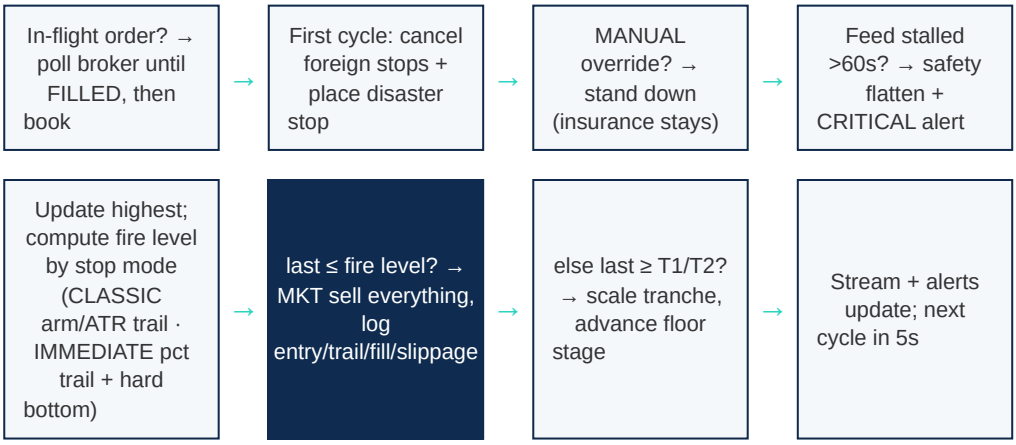
8 Safety invariants

Table 4 — Invariants of the exit engine — a change that breaks one is a regression

Invariant	Mechanism
Exits only	The module submits SELL orders against existing positions only. There is no code path that opens or adds.
Ratchets never move down	Trail and hard bottom are computed with `max()` against stored state; manual lowers are rejected and require MANUAL control instead.
Protection first	The fire check runs before scale-out evaluation every cycle — profit-taking never precedes protection.
One authority per position	Foreign broker-side stops are cancelled on first management; the disaster stop sits deliberately far below the working levels.
Kill switch wins	Checked every 5-second cycle per user; ON = the bot takes zero actions for that user.
UNKNOWN is a dead end	An unresolvable broker state is never resubmitted — position stays open, CRITICAL alert demands manual reconciliation, the position is excluded from retries.
Bookkeeping never blocks execution	Ledger writes are failure-isolated from broker calls; the alert trail preserves fill facts independently.
Off-hours honesty	The loop sleeps outside the session; no fabricated marks, no stall false-alarms at night. The disaster stop remains the overnight insurance.

9 Execution flow

One 5-second cycle for one position (market hours, kill switch off):



10 Database migration

All changes are applied automatically at boot by the guarded statements in Appendix D — no manual migration. Two batches:

10.1 Trailing state (v2.3)

- `positions.stopPrice` — original flat stop (governs pre-arm in CLASSIC; R reference everywhere).
- `positions.t1Price` / `highestPrice` / `trailPrice` / `trailArmed` / `scaleStage` / `closedBy` — trail state machine and close attribution.

10.2 Final update (v2.4)

- `users.stopMode VARCHAR(16) DEFAULT 'CLASSIC'` — per-user stop policy.
- `order_tickets.target2` — the second scale-out target, carried from engine proposals.
- `positions.t2Price`, `overrideMode` (null = module-managed, MANUAL = human-owned), `disasterOrderId`, `disasterPrice`.

> **DEPLOYMENT REMINDER**
>
> Unchanged from previous versions: the server only listens under `NODE_ENV=production node dist/boot.js`; delete stale `*.tsbuildinfo` before `tsc --noEmit`; if port 3000 answers with an old build, a respawned dev server is holding it. The trailing loop starts only in the production block — System Check reports "Trailing loop is NOT running" as a FAIL otherwise.

11 Implementation steps

Apply in this order. Appendices carry full source for new/replaced files and exact snippets for edits — everything is verbatim from the verified v2.4 build.

1. **Schema.** Appendix C into `db/schema.ts` (stopMode, target2, and the full positions column set).
2. **Boot migration.** Appendix D into `api/lib/ensure-schema.ts` (all guarded ALTER statements).
3. **Broker layer.** Appendix E — `BrokerOpenOrder` + optional `getOpenOrders` in `api/brokers/types.ts`, and the IBKR implementation.
4. **Portfolio.** Replace `api/engine/portfolio.ts` wholesale with Appendix B (supersedes the v2.2 file — adds trail seeding to recordFill, bookScaleOut, stop mode, overrides, activity stream).
5. **Trailing module.** Create `api/engine/trailing.ts` from Appendix A (new file — the core of this guide).
6. **T2 plumbing.** Appendix F into `api/queries/tickets.ts` and `api/engine/monitor.ts`.
7. **Router.** Appendix G into `api/engine-router.ts` (trailing status, stop mode, overrides, flatten, activity).
8. **System Check + boot.** Appendix H — the 12th probe into `api/systemcheck/checks.ts`, and `ensureTrailingLoop()` into the production block of `api/boot.ts`.
9. **UI.** Replace `src/components/OrdersPanel.tsx` with Appendix I; create `src/components/LiveStream.tsx` from Appendix J and mount it on the Autonomous page.
10. **Verify.** `rm -f *.tsbuildinfo && npx tsc --noEmit` (silent), `npm run build`, boot with `NODE_ENV=production`. Expect `schema ensured (16 tables)`, HTTP 401 on `engine.trailing` / `engine.getStopMode` / `engine.activity`, and 405 (POST-only) on the mutations.
11. **Exercise.** Run the Section 13 test plan against a paper account before any live-broker session.

12 API reference

Table 5 — tRPC routes added in v2.3 + v2.4 (all user-scoped, auth required)

Route	Kind	Behaviour
`engine.trailing`	query	Module status: running, marketHours, lastTickAt, managed count, in-flight exits, fires today.
`engine.getStopMode` / `setStopMode`	query / mutation	Per-user CLASSIC vs IMMEDIATE_TRAIL. Read fails to CLASSIC (conservative).
`engine.tightenStop`	mutation	`{ positionId, stopPrice }` — raise the flat stop. Rejects lowers and levels at/above the mark.
`engine.tightenTrail`	mutation	`{ positionId, trailPrice }` — raise the trail; arms it if unarmed.
`engine.setPositionOverride`	mutation	`{ positionId, manual }` — take/return manual control of the exit. Audited.
`engine.flattenPosition`	mutation	`{ positionId }` — immediate market flatten via the audited fire path, works off-hours.
`engine.activity`	query	Live stream feed — latest 40 execution-relevant events, newest first.

`engine.positions` rows now also carry `stopPrice`, `highestPrice`, `trailPrice`, `trailArmed`, `scaleStage`, `closedBy`, `t2Price`, `overrideMode`, `disasterPrice`. No route exposes the module's internal fire path — it is reachable only from the loop and the audited `requestFlatten`.

13 Test plan

Run against a paper account during market hours. Each step names the pass condition.

1. **Arm and ratchet (CLASSIC).** Open a small position with a governed stop. Pass: Protection column shows "flat" pre-arm; after price exceeds entry + 1R the TRAIL badge appears; the trail price only moves up on new highs, never down.
2. **Flat-stop fire.** On a position that never arms, walk price to the stop (paper). Pass: full market sell, position CLOSED with closedBy FLAT_STOP, alert shows entry / stop at fire / fill / slippage.
3. **Trail fire.** On an armed position, let price fall through the trail. Pass: closedBy TRAIL, slippage logged, disaster stop cancelled (no orphan), stream shows the event within seconds.
4. **Scale-outs.** Position of 6+ shares reaching T1. Pass: ~1/3 sold, realized P&L booked, floor badge shows BE FLOOR; at T2 another tranche, badge T1 FLOOR. 1–2 share positions skip scaling cleanly.
5. **IMMEDIATE_TRAIL.** Switch the Trail toggle; open a position. Pass: TRAIL badge from the first cycle at ~0.5% below price; distance tightens at the gain milestones; bottom reaches breakeven after +1% (a fade to entry closes at ~0, not a loss).
6. **Overrides.** TIGHTEN above current level → respected next cycle. TIGHTEN below → rejected with message. MANUAL → badge shows, module ignores the position (no fire even through the trail); AUTO → protection resumes. FLATTEN → immediate market sell with audit.
7. **Cancel-and-govern.** Place a manual STP sell at the broker for a managed symbol. Pass: within one cycle it is cancelled and a confirmation alert names the order.
8. **Disaster stop.** On a live-broker position: +D badge appears, order visible at the broker far below; close the position → the resting order disappears. On paper: correctly skipped.
9. **Feed stall.** Stop the data gateway with a position open. Pass: within ~75s a CRITICAL feed-stall alert and a flatten attempt; broker unreachable → follow-up "flatten manually".
10. **Kill switch.** Flip the global kill switch with positions open. Pass: all module action stops immediately; resumes when switched back.
11. **System Check.** Manual run from the owner page: the Trailing-stop module probe is OK and reports the managed count.

Appendix A — api/engine/trailing.ts (new file, full source)

The complete protective-exit engine, v2.4: classic flat-stop → 1R arm → 2.5×ATR trail, immediate-trail mode with hard bottom, T1/T2 scale-outs, fire locks, broker-side reconciliation, disaster stop, safety flatten, and kill-switch checks every cycle. Create as a new file.

api/engine/trailing.ts — create as a new file

```
import { and, eq } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { alerts, orderTickets, positions, type Position } from "@db/schema";
import { etParts } from "../marketdata/indicators";
import { isMarketHours, marketDataService } from "../marketdata/service";
import { resolveBroker, type BrokerCode } from "../brokers/registry";
import type { BrokerAdapter, OrderIntent } from "../brokers/types";
import { getAiLimits } from "../queries/autonomous";
import { atr } from "./triggers";
import {
  bookScaleOut,
  closePositionFromFire,
  getStopMode,
  listManagedPositions,
  updateTrailState,
  type StopMode,
} from "./portfolio";

/**
 * TRAILING MODULE v2 — the complete automatic exit engine.
 *
 * Manages any OPEN position that carries a governed stop (all engine and
 * Intelligence tickets do), regardless of entry path. Exits only — this
 * module can never open or add to a position.
 *
 * STOP MODES (per user, users.stopMode):
 * - CLASSIC        — flat stop until price reaches entry + 1R, then a
 *                    2.5×ATR14 trail off the highest price, ratchet-only,
 *                    with breakeven (post-T1) and T1 (post-T2) floors.
 * - IMMEDIATE_TRAIL — no flat stop at all: a percentage trail is live from
 *                    entry at the same 0.5% margin the flat stop would have
 *                    used, tightening on gain milestones (0.50% → 0.45 → 0.40
 *                    → 0.32 → 0.27 → 0.22 floor, with a 0.5×ATR14 distance
 *                    floor for jumpy names), plus a HARD BOTTOM that starts
 *                    at entry − 0.5%, moves to breakeven at +1%, and locks
 *                    40% of peak gain beyond +2%. Fire = last ≤ max(trail,
 *                    bottom). No broker-side stop-loss is used anywhere.
 *
 * SCALE-OUTS: at T1 (and T2 when the proposal carried one) the module sells
 * a tranche at market — 1/3 then 1/2 of the remainder — booking realized
 * P&L and advancing the floor stage. Positions smaller than 3 shares skip
 * scaling; the trail runs the whole position.
 *
 * BROKER-SIDE RECONCILIATION (once per position, first management cycle):
 * - Cancel-and-govern: any resting SELL stop on the same symbol that we did
 *   not create is cancelled, with a confirmation alert — the module is the
 *   single exit authority.
 * - Disaster stop: one catastrophic broker-side STP is placed far below the
 *   flat stop (buffer = max(1% of entry, ATR14)) as pure crash insurance,
 *   polled once a minute, cancelled when the position closes. Skipped on
 *   the paper adapter (which fills resting orders instantly).
 *
 * SAFETY: feed stall > one refresh cycle (>60s) → attempt to flatten AND
 * raise a CRITICAL alert (throttled 5 min per position). The per-user kill
 * switch is checked every cycle — when ON, the bot stands down entirely.
 * UNKNOWN broker states are dead ends (manual reconciliation, never
 * resubmitted). Off-hours the module sleeps; positions rest on their stops.
```

```
 */

const TICK_MS = 5_000;
const STALL_MS = 75_000;
const RETRY_MS = 60_000;
const SAFETY_THROTTLE_MS = 300_000;
const DISASTER_POLL_MS = 60_000;
const ARM_R = 1.0;
const TRAIL_ATR_MULT = 2.5;

/* IMMEDIATE_TRAIL schedule */
const IT_INITIAL_PCT = 0.005; // same margin the flat stop would use
const IT_MIN_PCT = 0.0022; // never tighter than this — ordinary noise must not kill a runner
const IT_ATR_FLOOR_MULT = 0.5; // distance never below 0.5×ATR14
const IT_BREAKEVEN_GAIN = 0.01; // hard bottom → entry at +1%
const IT_LOCKIN_GAIN = 0.02; // beyond +2% the bottom locks 40% of peak gain
const IT_LOCKIN_FRAC = 0.4;

type ExitReason = "FLAT_STOP" | "TRAIL" | "SAFETY_FLATTEN" | "MANUAL_FLATTEN" | "SCALE_T1" | "SCALE_T2";

interface FireLock { at: number; orderId: string | null; reason: ExitReason; qty: number; ref: number }
const firing = new Map<number, FireLock>();
const deadEnded = new Set<number>();
const safetyAt = new Map<number, number>();
const reconciled = new Set<number>();
const disasterPollAt = new Map<number, number>();

let loopStarted = false;
let ticking = false;
let lastTickAt: number | null = null;
let lastManagedCount = 0;
let firedDay = "";
let firedToday = 0;

export function trailingStatus(): { running: boolean; marketHours: boolean; lastTickAt: number | null;
managed: number; inFlight: number; firedToday: number } {
  return { running: loopStarted, marketHours: isMarketHours(), lastTickAt, managed: lastManagedCount,
inFlight: firing.size, firedToday };
}

export function ensureTrailingLoop(): void {
  if (loopStarted) return;
  loopStarted = true;
  const t = setInterval(() => { void tick(); }, TICK_MS);
  t.unref();
}

/** Manual flatten from the UI — works any time, even off-hours. Same broker path, same audit. */
export async function requestFlatten(userId: number, positionId: number): Promise<{ ok: boolean; message:
string }> {
  const db = getDb();
  const [p] = await db.select().from(positions)
    .where(and(eq(positions.id, positionId), eq(positions.userId, userId), eq(positions.status,
"OPEN"))).limit(1);
  if (!p) return { ok: false, message: "No open position with that id." };
  const last = marketDataService.get(p.symbol)?.indicators?.last ?? Number(p.avgEntry);
  const ref = p.trailPrice !== null ? Number(p.trailPrice) : p.stopPrice !== null ? Number(p.stopPrice) :
Number(p.avgEntry);
  await submitExit(userId, p, "MANUAL_FLATTEN", p.quantity, ref, last);
  return { ok: true, message: `Flatten order submitted for ${p.symbol} (${p.quantity} sh at market) —
watch the stream for the fill.` };
}

async function tick(): Promise<void> {
  if (ticking) return;
  ticking = true;
  try {
    if (!isMarketHours()) return; // sleeps off-hours — positions rest on their stops
```

```
    lastTickAt = Date.now();
    const day = etParts(lastTickAt).day;
    if (day !== firedDay) { firedDay = day; firedToday = 0; }

    const rows = await listManagedPositions();
    lastManagedCount = rows.length;
    const byUser = new Map<number, Position[]>();
    for (const p of rows) {
      const list = byUser.get(p.userId) ?? [];
      list.push(p);
      byUser.set(p.userId, list);
    }
    for (const [userId, list] of byUser) {
      // KILL SWITCH — checked every cycle. When ON the human owns every exit.
      const limits = await getAiLimits(userId).catch(() => null);
      if (limits?.killSwitch === true) continue;
      const mode = await getStopMode(userId);
      for (const p of list) await evaluate(userId, p, mode).catch(() => undefined);
    }
  } finally {
    ticking = false;
  }
}

/* ---------- broker resolution (the position's own broker, from its source ticket) ---------- */

async function brokerFor(userId: number, p: Position): Promise<{ adapter: BrokerAdapter; effective:
string; accountId: string }> {
  const db = getDb();
  const [ticket] = p.sourceTicketId
    ? await db.select().from(orderTickets).where(and(eq(orderTickets.userId, userId),
eq(orderTickets.ticketId, p.sourceTicketId))).limit(1)
    : [undefined];
  const { adapter, effective } = resolveBroker(((ticket?.broker as BrokerCode | undefined) ?? "PAPER"));
  const accountId = ticket?.accountId ?? (await adapter.getAccounts().catch(() => []))[0]?.accountId ??
"PAPER-001";
  return { adapter, effective, accountId };
}

/* ---------- evaluation ---------- */

async function evaluate(userId: number, p: Position, mode: StopMode): Promise<void> {
  // An in-flight exit/scale order owns the position until the broker resolves it.
  const lock = firing.get(p.id);
  if (lock) {
    if (lock.orderId) await pollInFlight(userId, p, lock).catch(() => undefined);
    else if (Date.now() - lock.at > RETRY_MS) firing.delete(p.id);
    return;
  }
  if (deadEnded.has(p.id)) return;

  const { adapter, effective, accountId } = await brokerFor(userId, p);

  // Once per position: cancel foreign stops, place the disaster stop.
  if (!reconciled.has(p.id)) {
    reconciled.add(p.id);
    await reconcileBrokerSide(userId, p, adapter, effective, accountId).catch(() => undefined);
  }

  // Disaster-stop poll (once a minute): if the catastrophe order filled, the broker closed the position.
  if (p.disasterOrderId && (Date.now() - (disasterPollAt.get(p.id) ?? 0)) > DISASTER_POLL_MS) {
    disasterPollAt.set(p.id, Date.now());
    await pollDisaster(userId, p, adapter, effective, accountId).catch(() => undefined);
    const [fresh] = await getDb().select().from(positions).where(eq(positions.id, p.id)).limit(1).catch(()
=> []);
    if (!fresh || fresh.status !== "OPEN") return;
    p = fresh;
  }
```

```
  // MANUAL override — the human owns this exit. (Disaster insurance above stays active.)
  if (p.overrideMode === "MANUAL") return;

  const feed = marketDataService.get(p.symbol);
  const last = feed?.indicators?.last ?? null;
  const stalled = !feed
    || feed.lastRefresh === null
    || Date.now() - feed.lastRefresh > STALL_MS
    || feed.error !== null
    || last === null;

  if (stalled) {
    await safetyFlatten(userId, p, last ?? Number(p.avgEntry));
    return;
  }

  const entry = Number(p.avgEntry);
  const stop = Number(p.stopPrice);
  const risk = entry - stop;
  if (!(risk > 0)) return;

  const atr14 = atr(feed.bars, 14) ?? entry * 0.005;
  const highest = Math.max(p.highestPrice !== null ? Number(p.highestPrice) : entry, last);

  let armed = p.trailArmed === true;
  let trail = p.trailPrice !== null ? Number(p.trailPrice) : null;
  let fireLevel: number;
  let fireReason: ExitReason;

  if (mode === "IMMEDIATE_TRAIL") {
    // Trail is live from entry; distance tightens on gain milestones.
    armed = true;
    const peakGainPct = (highest - entry) / entry;
    const distPct = immediateDistPct(peakGainPct);
    const distance = Math.max(distPct * highest, IT_ATR_FLOOR_MULT * atr14);
    const candidate = highest - distance;
    trail = Math.max(trail ?? Number.NEGATIVE_INFINITY, candidate);
    trail = +trail.toFixed(4);
    // HARD BOTTOM — deterministic ratchet off the highest price.
    let bottom = entry - IT_INITIAL_PCT * entry;
    if (peakGainPct >= IT_BREAKEVEN_GAIN) bottom = Math.max(bottom, entry);
    if (peakGainPct >= IT_LOCKIN_GAIN) bottom = Math.max(bottom, entry + IT_LOCKIN_FRAC * (highest -
entry));
    fireLevel = Math.max(trail, bottom);
    fireReason = "TRAIL";
  } else {
    // CLASSIC — flat stop governs until entry + 1R, then the ATR trail.
    if (!armed && last >= entry + ARM_R * risk) {
      armed = true;
      trail = highest - TRAIL_ATR_MULT * atr14;
    }
    if (armed) {
      const floor = p.scaleStage >= 2 && p.t1Price !== null
        ? Number(p.t1Price)
        : p.scaleStage >= 1 ? entry : null;
      const candidate = highest - TRAIL_ATR_MULT * atr14;
      const next = Math.max(trail ?? Number.NEGATIVE_INFINITY, candidate, floor ??
Number.NEGATIVE_INFINITY);
      trail = Number.isFinite(next) ? +next.toFixed(4) : trail;
    }
    fireLevel = armed && trail !== null ? trail : stop;
    fireReason = armed ? "TRAIL" : "FLAT_STOP";
  }

  // persist progress when anything moved
  const changed =
    highest !== (p.highestPrice !== null ? Number(p.highestPrice) : null)
```

```
    || armed !== (p.trailArmed === true)
    || trail !== (p.trailPrice !== null ? Number(p.trailPrice) : null);
  if (changed) {
    await updateTrailState(p.id, {
      highestPrice: highest.toFixed(4),
      trailArmed: armed,
      ...(trail !== null ? { trailPrice: trail.toFixed(4) } : {}),
    }).catch(() => undefined);
  }

  // 1. Protection first — the fire check always wins.
  if (last <= fireLevel) {
    await submitExit(userId, p, fireReason, p.quantity, fireLevel, last);
    return;
  }

  // 2. Scale-outs on strength — T1 sells a third, T2 sells half the remainder.
  if (p.quantity >= 3) {
    const t1 = p.t1Price !== null ? Number(p.t1Price) : null;
    const t2 = p.t2Price !== null ? Number(p.t2Price) : null;
    if (p.scaleStage === 0 && t1 !== null && last >= t1) {
      await submitExit(userId, p, "SCALE_T1", Math.max(1, Math.round(p.quantity / 3)), t1, last);
    } else if (p.scaleStage === 1 && t2 !== null && last >= t2) {
      await submitExit(userId, p, "SCALE_T2", Math.max(1, Math.floor(p.quantity / 2)), t2, last);
    }
  }
}

/** IMMEDIATE_TRAIL distance schedule: 0.50% → 0.45 → 0.40 → 0.32 → 0.27 → 0.22 (floor). */
function immediateDistPct(peakGainPct: number): number {
  let d: number;
  if (peakGainPct <= 0.01) d = 0.005 - 0.1 * peakGainPct;
  else if (peakGainPct <= 0.015) d = 0.004 - 0.16 * (peakGainPct - 0.01);
  else d = 0.0032 - 0.1 * (peakGainPct - 0.015);
  return Math.max(IT_MIN_PCT, Math.min(IT_INITIAL_PCT, d));
}

/* ---------- order submission ---------- */

/** Market-sell qty through the position's own broker. Full exits and scale tranches share this path. */
async function submitExit(userId: number, p: Position, reason: ExitReason, qty: number, refPrice: number,
last: number): Promise<void> {
  if (firing.has(p.id) || deadEnded.has(p.id)) return;
  firing.set(p.id, { at: Date.now(), orderId: null, reason, qty, ref: refPrice });

  const { adapter, effective, accountId } = await brokerFor(userId, p);
  const intent: OrderIntent = {
    symbol: p.symbol,
    side: "SELL",
    quantity: qty,
    orderType: "MKT",
    tif: "DAY",
    clientOrderId: `TRAIL-${p.id}-${Date.now()}`,
    // the paper simulator fills at limitPrice — pass the mark so paper bookkeeping stays honest;
    // live brokers get a pure market order (no price fields attached)
    ...(adapter.code === "PAPER" ? { limitPrice: last } : {}),
  };

  let ack;
  try {
    ack = await adapter.placeOrder(accountId, intent);
  } catch (e) {
    firing.delete(p.id);
    await alertFire(userId, p, "CRITICAL",
      `PROTECTIVE EXIT FAILED — flatten manually`,
      `${reason} for ${p.symbol} ${qty} sh could not reach the broker (${(e as Error).message.slice(0,
160)}). entry ${Number(p.avgEntry).toFixed(2)} · reference ${refPrice.toFixed(2)} · last known
${last.toFixed(2)}. The position is still OPEN — flatten it manually now.`);
```

```
    return;
  }

  if (ack.status === "UNKNOWN") {
    firing.delete(p.id);
    deadEnded.add(p.id); // dead end — never auto-retry an unknown broker state
    await alertFire(userId, p, "CRITICAL",
      `EXIT STATE UNKNOWN — manual reconciliation required`,
      `${reason} for ${p.symbol} ${qty} sh: broker returned an unresolvable state (${(ack.message ??
"").slice(0, 140)}). The order was NOT resubmitted. Reconcile with the broker, then close position
#${p.id} manually.`);
    return;
  }
  if (ack.status === "REJECTED") {
    firing.delete(p.id);
    await alertFire(userId, p, "HIGH",
      `${reason === "SCALE_T1" || reason === "SCALE_T2" ? "Scale-out" : "Protective exit"} rejected —
retrying`,
      `${reason} for ${p.symbol} ${qty} sh rejected by ${effective}: ${(ack.message ?? "no reason
given").slice(0, 140)}. The module will retry on the next cycle.`);
    return;
  }

  if (ack.status === "FILLED") {
    const fill = ack.averagePrice !== undefined ? Number(ack.averagePrice) : last;
    firing.delete(p.id);
    await bookExit(userId, p, reason, qty, refPrice, fill, ack.brokerOrderId, effective);
    return;
  }

  // ACKNOWLEDGED / WORKING — hold the fire lock and poll for the real fill.
  firing.set(p.id, { at: Date.now(), orderId: ack.brokerOrderId, reason, qty, ref: refPrice });
  await alertFire(userId, p, reason === "SAFETY_FLATTEN" || reason === "MANUAL_FLATTEN" ? "CRITICAL" :
"HIGH",
    `${reasonLabel(reason)} fired — order working`,
    `${reason} for ${p.symbol} ${qty} sh acknowledged by ${effective} (order ${ack.brokerOrderId}) —
booking when the broker reports the fill. entry ${Number(p.avgEntry).toFixed(2)} · reference at fire
${refPrice.toFixed(2)} · trigger price ${last.toFixed(2)}.`);
}

/** Poll a working exit order until the broker reports the fill, then book it. */
async function pollInFlight(userId: number, p: Position, lock: FireLock): Promise<void> {
  const { adapter, effective, accountId } = await brokerFor(userId, p);
  const status = await adapter.getOrderStatus(accountId, lock.orderId ?? "");
  if (status.status === "FILLED") {
    const fill = status.averagePrice !== undefined ? Number(status.averagePrice) : lock.ref;
    firing.delete(p.id);
    await bookExit(userId, p, lock.reason, lock.qty, lock.ref, fill, lock.orderId ?? "", effective);
  } else if (status.status === "REJECTED" || status.status === "UNKNOWN") {
    firing.delete(p.id);
    if (status.status === "UNKNOWN") deadEnded.add(p.id);
    await alertFire(userId, p, "CRITICAL",
      `Working exit ${status.status.toLowerCase()} — manual reconciliation required`,
      `${p.symbol} ${lock.qty} sh exit order ${lock.orderId} is now ${status.status} (${(status.message ??
"").slice(0, 140)}). Position #${p.id} remains OPEN — reconcile with the broker and flatten manually if
needed.`);
  }
  // still WORKING → keep the lock, poll again next cycle
}

/** Book a fill: full exits close the row (and cancel the disaster stop); scale tranches split it. */
async function bookExit(userId: number, p: Position, reason: ExitReason, qty: number, refPrice: number,
fill: number, brokerOrderId: string, effective: string): Promise<void> {
  if (reason === "SCALE_T1" || reason === "SCALE_T2") {
    const stage = reason === "SCALE_T1" ? 1 : 2;
    await bookScaleOut(p.id, qty, fill, stage as 1 | 2);
    firedToday += 1;
    const pnl = ((fill - Number(p.avgEntry)) * qty).toFixed(2);
```

```
    await alertFire(userId, p, "HIGH",
      `Scale-out ${stage === 1 ? "T1" : "T2"} filled — ${p.symbol}`,
      `${reason} · ${p.symbol} SOLD ${qty} @ ${fill.toFixed(2)} via ${effective} (order ${brokerOrderId})
· entry ${Number(p.avgEntry).toFixed(2)} · target ${refPrice.toFixed(2)} · realized P&L ${Number(pnl) >= 0
? "+" : ""}${pnl} · trailing floor now ${stage === 1 ? "BREAKEVEN" : "T1 PRICE"} · trail runs the
remainder.`);
    return;
  }

  await closePositionFromFire(p.id, fill, reason === "MANUAL_FLATTEN" ? "MANUAL_FLATTEN" : reason ===
"SAFETY_FLATTEN" ? "SAFETY_FLATTEN" : reason);
  firedToday += 1;
  const slippage = +(fill - refPrice).toFixed(4);
  const pnl = ((fill - Number(p.avgEntry)) * qty).toFixed(2);
  await alertFire(userId, p, reason === "SAFETY_FLATTEN" || reason === "MANUAL_FLATTEN" ? "CRITICAL" :
"HIGH",
    `${reasonLabel(reason)} — ${p.symbol} closed`,
    `${reason} · ${p.symbol} SOLD ${qty} @ ${fill.toFixed(2)} via ${effective} (order ${brokerOrderId}) ·
entry ${Number(p.avgEntry).toFixed(2)} · reference at fire ${refPrice.toFixed(2)} · fill
${fill.toFixed(2)} · slippage ${slippage >= 0 ? "+" : ""}${slippage} · realized P&L ${Number(pnl) >= 0 ?
"+" : ""}${pnl}.`);

  // the disaster stop's job is done — never leave orphan resting orders
  if (p.disasterOrderId) {
    const { adapter, accountId } = await brokerFor(userId, p);
    await adapter.cancelOrder(accountId, p.disasterOrderId).catch(() => undefined);
    await getDb().update(positions).set({ disasterOrderId: null }).where(eq(positions.id, p.id)).catch(()
=> undefined);
  }
}

/* ---------- broker-side reconciliation (once per position) ---------- */

async function reconcileBrokerSide(userId: number, p: Position, adapter: BrokerAdapter, effective: string,
accountId: string): Promise<void> {
  // CANCEL-AND-GOVERN — remove any resting sell-stop we did not create.
  if (adapter.getOpenOrders) {
    try {
      const open = await adapter.getOpenOrders(accountId);
      const foreign = open.filter((o) =>
        o.side === "SELL"
        && /STP|STOP|TRAIL/.test(o.orderType)
        && o.symbol === p.symbol
        && o.brokerOrderId !== p.disasterOrderId
        && !(o.clientOrderId ?? "").startsWith("TRAIL-")
        && !(o.clientOrderId ?? "").startsWith("DSTR-"));
      for (const o of foreign) {
        const res = await adapter.cancelOrder(accountId, o.brokerOrderId).catch((e) => ({ ok: false,
message: (e as Error).message }));
        await alertFire(userId, p, res.ok ? "HIGH" : "CRITICAL",
          res.ok ? `Broker-side stop canceled — trailing module governs ${p.symbol}` : `Could NOT cancel
broker-side stop on ${p.symbol}`,
          res.ok
            ? `Resting ${o.orderType} sell order ${o.brokerOrderId}${o.stopPrice !== null ? ` @
${o.stopPrice}` : ""} on ${p.symbol} was canceled so it cannot preempt the trailing module. The module now
governs this exit${p.disasterPrice !== null ? ` (disaster insurance remains far below)` : ""}.`
            : `Resting ${o.orderType} sell order ${o.brokerOrderId} on ${p.symbol} could not be canceled
(${res.message.slice(0, 120)}). It may fire before the trailing module — cancel it manually to avoid a
double exit.`);
      }
    } catch (e) {
      await alertFire(userId, p, "HIGH",
        `Could not inspect broker-side orders for ${p.symbol}`,
        `getOpenOrders failed (${(e as Error).message.slice(0, 140)}). If you placed a stop manually at
${effective}, cancel it — the trailing module is the exit authority for this position.`);
    }
  } else if (adapter.code !== "PAPER") {
    await alertFire(userId, p, "MEDIUM",
```

```
      `Broker does not expose open orders — manual check`,
      `${effective} cannot list resting orders, so the module cannot verify no broker-side stop exists on
${p.symbol}. If you placed one manually, cancel it to avoid a double exit.`);
  }

  // DISASTER STOP — one catastrophic resting STP far below, pure crash insurance.
  if (adapter.code !== "PAPER" && !p.disasterOrderId && p.stopPrice !== null) {
    const entry = Number(p.avgEntry);
    const stop = Number(p.stopPrice);
    const feed = marketDataService.get(p.symbol);
    const atr14 = (feed ? atr(feed.bars, 14) : null) ?? entry * 0.005;
    const disaster = +(stop - Math.max(0.01 * entry, atr14)).toFixed(2);
    if (disaster > 0) {
      try {
        const ack = await adapter.placeOrder(accountId, {
          symbol: p.symbol, side: "SELL", quantity: p.quantity, orderType: "STP",
          stopPrice: disaster, tif: "GTC", clientOrderId: `DSTR-${p.id}`,
        });
        if (ack.brokerOrderId && ack.status !== "REJECTED" && ack.status !== "UNKNOWN") {
          await getDb().update(positions).set({ disasterOrderId: ack.brokerOrderId, disasterPrice:
String(disaster) }).where(eq(positions.id, p.id));
          await alertFire(userId, p, "LOW",
            `Disaster stop resting — ${p.symbol}`,
            `Catastrophe insurance placed on ${p.symbol}: STP sell ${p.quantity} @ ${disaster} GTC via
${effective} (order ${ack.brokerOrderId}). Far below the working stop (${stop}) — it only fires if the
module and its safety net both fail. Canceled automatically when the position closes.`);
        } else {
          await alertFire(userId, p, "MEDIUM", `Disaster stop could not rest — ${p.symbol}`, `${effective}
rejected or could not place the catastrophe stop @ ${disaster} (${(ack.message ?? "").slice(0, 120)}). The
module still governs the exit; you simply have no broker-side insurance layer.`);
        }
      } catch (e) {
        await alertFire(userId, p, "MEDIUM", `Disaster stop placement failed — ${p.symbol}`, `${(e as
Error).message.slice(0, 140)}. The module still governs the exit.`);
      }
    }
  }
}

/** If the disaster stop filled at the broker, the position is gone — close the books honestly. */
async function pollDisaster(userId: number, p: Position, adapter: BrokerAdapter, effective: string,
accountId: string): Promise<void> {
  const status = await adapter.getOrderStatus(accountId, p.disasterOrderId ?? "");
  const db = getDb();
  if (status.status === "FILLED") {
    const fill = status.averagePrice !== undefined ? Number(status.averagePrice) : Number(p.disasterPrice
?? p.stopPrice);
    await closePositionFromFire(p.id, fill, "DISASTER_STOP");
    await db.update(positions).set({ disasterOrderId: null }).where(eq(positions.id, p.id)).catch(() =>
undefined);
    firedToday += 1;
    await alertFire(userId, p, "CRITICAL",
      `DISASTER STOP FILLED — ${p.symbol} closed at broker`,
      `The catastrophe stop on ${p.symbol} fired @ ${fill.toFixed(2)} via ${effective} (order
${p.disasterOrderId}). This means price collapsed far below the working stop (${Number(p.stopPrice ??
0).toFixed(2)}) faster than the module could act, or the module was impaired. entry
${Number(p.avgEntry).toFixed(2)} · realized P&L booked. Investigate the feed/module before the next
session.`);
  } else if (/cancel/i.test(status.status) || status.status === "REJECTED") {
    // insurance vanished — clear it so reconciliation re-places it next session
    await db.update(positions).set({ disasterOrderId: null, disasterPrice: null }).where(eq(positions.id,
p.id)).catch(() => undefined);
    reconciled.delete(p.id);
    await alertFire(userId, p, "MEDIUM", `Disaster stop gone — ${p.symbol}`, `The resting catastrophe
order ${p.disasterOrderId} is ${status.status}. The module will attempt to re-place insurance; the working
stop is unaffected.`);
  }
}
```

```
/* ---------- safety ---------- */

async function safetyFlatten(userId: number, p: Position, reference: number): Promise<void> {
  const prev = safetyAt.get(p.id) ?? 0;
  if (Date.now() - prev < SAFETY_THROTTLE_MS) return;
  safetyAt.set(p.id, Date.now());
  await alertFire(userId, p, "CRITICAL",
    `FEED STALL — attempting safety flatten`,
    `${p.symbol} ${p.quantity} sh: price feed stalled for more than one refresh cycle (>60s). Attempting
to flatten the position now; if the broker is unreachable you will get a follow-up alert — flatten
manually in that case.`);
  await submitExit(userId, p, "SAFETY_FLATTEN", p.quantity, reference, reference);
}

/* ---------- helpers ---------- */

function reasonLabel(reason: ExitReason): string {
  switch (reason) {
    case "TRAIL": return "Trailing stop";
    case "FLAT_STOP": return "Flat stop";
    case "SAFETY_FLATTEN": return "SAFETY FLATTEN";
    case "MANUAL_FLATTEN": return "Manual flatten";
    case "SCALE_T1": return "Scale-out T1";
    case "SCALE_T2": return "Scale-out T2";
  }
}

async function alertFire(userId: number, p: Position, priority: "CRITICAL" | "HIGH" | "MEDIUM" | "LOW",
title: string, body: string): Promise<void> {
  const db = getDb();
  await db.insert(alerts).values({
    userId,
    type: "EXECUTION",
    priority,
    title,
    body,
    symbol: p.symbol,
    state: "ACTIVE",
  }).catch(() => undefined);
}
```

Appendix B — api/engine/portfolio.ts (full replacement, v2.4)

Replaces the v2.2 file in its entirety. Adds trail-state bookkeeping (`updateTrailState`, `listManagedPositions`), scale-out ledgering (`bookScaleOut`), fire closes (`closePositionFromFire` with DISASTER_STOP / MANUAL_FLATTEN reasons), stop-mode preference, override controls (`tightenStop`, `tightenTrail`, `setPositionOverride`), and the live-stream projection (`streamActivity`).

api/engine/portfolio.ts — replace the v2.2 file

```
import { and, desc, eq, ne, sql } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { positions, orderTickets, users, alerts, type OrderTicket } from "@db/schema";
import { marketDataService } from "../marketdata/service";

/**
 * PORTFOLIO — order history, positions, and P&L.
 *
 * Every fill (manual CONFIRM or auto-execute) lands here via recordFill:
 *   BUY  → open a position (or average into an existing open one)
 *   SELL → close matching open position(s), booking realized P&L
 *
 * Unrealized P&L marks open positions to the live feed's last price when
 * available, and honestly reports when a mark is unavailable (no feed)
 * instead of inventing a price.
 */

/* ---------- auto-execute preference ---------- */

export async function isAutoExecuteEnabled(userId: number): Promise<boolean> {
  try {
    const db = getDb();
    const [u] = await db.select({ autoExecute: users.autoExecute }).from(users).where(eq(users.id,
userId)).limit(1);
    return u?.autoExecute === true;
  } catch {
    return false; // fail CLOSED — if we can't read the flag, confirmation is required
  }
}

export async function setAutoExecute(userId: number, enabled: boolean): Promise<{ enabled: boolean }> {
  const db = getDb();
  await db.update(users).set({ autoExecute: enabled }).where(eq(users.id, userId));
  return { enabled };
}

/* ---------- fill recording ---------- */

export async function recordFill(userId: number, ticket: OrderTicket, fillPrice: number, fillQty: number):
Promise<void> {
  if (!(fillPrice > 0) || fillQty <= 0) return;
  const db = getDb();
  try {
    if (ticket.side === "BUY") {
      const [open] = await db
        .select()
        .from(positions)
        .where(and(eq(positions.userId, userId), eq(positions.symbol, ticket.symbol), eq(positions.status,
"OPEN")))
        .limit(1);
      if (open) {
        const totalQty = open.quantity + fillQty;
        const newAvg = (Number(open.avgEntry) * open.quantity + fillPrice * fillQty) / totalQty;
        const highest = Math.max(open.highestPrice !== null ? Number(open.highestPrice) : 0, fillPrice);
        await db.update(positions).set({ quantity: totalQty, avgEntry: newAvg.toFixed(4), highestPrice:
```

```
highest.toFixed(4) }).where(eq(positions.id, open.id));
      } else {
        await db.insert(positions).values({
          userId,
          symbol: ticket.symbol,
          quantity: fillQty,
          avgEntry: String(fillPrice),
          sourceTicketId: ticket.ticketId,
          status: "OPEN",
          // seed the trailing-stop state from the ticket's governed levels
          stopPrice: ticket.stop !== null ? String(ticket.stop) : null,
          t1Price: ticket.target !== null ? String(ticket.target) : null,
          t2Price: ticket.target2 !== null ? String(ticket.target2) : null,
          highestPrice: String(fillPrice),
        });
      }
    } else {
      // SELL — close against the oldest open position for the symbol
      const [open] = await db
        .select()
        .from(positions)
        .where(and(eq(positions.userId, userId), eq(positions.symbol, ticket.symbol), eq(positions.status,
"OPEN")))
        .limit(1);
      if (!open) return; // sell without a tracked position — nothing to match (manual external position)
      const closingQty = Math.min(fillQty, open.quantity);
      const realized = (fillPrice - Number(open.avgEntry)) * closingQty;
      if (closingQty >= open.quantity) {
        await db.update(positions).set({
          status: "CLOSED",
          closedAt: new Date(),
          exitPrice: String(fillPrice),
          realizedPnl: realized.toFixed(2),
        }).where(eq(positions.id, open.id));
      } else {
        // partial close = scale-out → advance the trailing floor stage on the remainder
        await db.update(positions).set({ quantity: open.quantity - closingQty, scaleStage: open.scaleStage
+ 1 }).where(eq(positions.id, open.id));
        await db.insert(positions).values({
          userId,
          symbol: open.symbol,
          quantity: closingQty,
          avgEntry: open.avgEntry,
          sourceTicketId: ticket.ticketId,
          status: "CLOSED",
          closedAt: new Date(),
          exitPrice: String(fillPrice),
          realizedPnl: realized.toFixed(2),
        });
      }
    }
  } catch {
    /* position tracking must never break execution */
  }
}

/* ---------- order history ---------- */

export interface OrderHistoryRow {
  ticketId: string;
  symbol: string;
  side: string;
  quantity: number;
  orderType: string;
  state: string;
  autoExecuted: boolean;
  entry: number | null;
  stop: number | null;
```

```
  target: number | null;
  filledQuantity: number | null;
  averageFillPrice: number | null;
  createdAt: Date;
  submittedAt: Date | null;
  lastMessage: string | null;
}

export async function listOrders(userId: number, limit = 50): Promise<OrderHistoryRow[]> {
  const db = getDb();
  const rows = await db
    .select()
    .from(orderTickets)
    .where(and(eq(orderTickets.userId, userId), ne(orderTickets.state, "CREATED")))
    .orderBy(desc(orderTickets.id))
    .limit(Math.min(limit, 200));
  return rows.map((t) => ({
    ticketId: t.ticketId,
    symbol: t.symbol,
    side: t.side,
    quantity: t.quantity,
    orderType: t.orderType,
    state: t.state,
    autoExecuted: t.autoExecuted === true,
    entry: t.entry !== null ? Number(t.entry) : null,
    stop: t.stop !== null ? Number(t.stop) : null,
    target: t.target !== null ? Number(t.target) : null,
    filledQuantity: t.filledQuantity ?? null,
    averageFillPrice: t.averageFillPrice !== null ? Number(t.averageFillPrice) : null,
    createdAt: t.createdAt,
    submittedAt: t.submittedAt ?? null,
    lastMessage: t.lastMessage ?? null,
  }));
}

/* ---------- positions & P&L ---------- */

export interface PositionRow {
  id: number;
  symbol: string;
  quantity: number;
  avgEntry: number;
  status: string;
  openedAt: Date;
  closedAt: Date | null;
  exitPrice: number | null;
  realizedPnl: number | null;
  mark: number | null; // live last price when the feed has one
  unrealizedPnl: number | null; // null when no mark available
  /* trailing-stop state */
  stopPrice: number | null;
  highestPrice: number | null;
  trailPrice: number | null;
  trailArmed: boolean;
  scaleStage: number;
  closedBy: string | null;
  t2Price: number | null;
  overrideMode: string | null;
  disasterPrice: number | null;
}

function toPositionRow(p: typeof positions.$inferSelect): PositionRow {
  const feed = marketDataService.get(p.symbol);
  const mark = feed?.indicators?.last ?? null;
  const isOpen = p.status === "OPEN";
  return {
    id: Number(p.id),
    symbol: p.symbol,
```

```
    quantity: p.quantity,
    avgEntry: Number(p.avgEntry),
    status: p.status,
    openedAt: p.openedAt,
    closedAt: p.closedAt ?? null,
    exitPrice: p.exitPrice !== null ? Number(p.exitPrice) : null,
    realizedPnl: p.realizedPnl !== null ? Number(p.realizedPnl) : null,
    mark,
    unrealizedPnl: isOpen && mark !== null ? +((mark - Number(p.avgEntry)) * p.quantity).toFixed(2) :
null,
    stopPrice: p.stopPrice !== null ? Number(p.stopPrice) : null,
    highestPrice: p.highestPrice !== null ? Number(p.highestPrice) : null,
    trailPrice: p.trailPrice !== null ? Number(p.trailPrice) : null,
    trailArmed: p.trailArmed === true,
    scaleStage: p.scaleStage,
    closedBy: p.closedBy ?? null,
    t2Price: p.t2Price !== null ? Number(p.t2Price) : null,
    overrideMode: p.overrideMode ?? null,
    disasterPrice: p.disasterPrice !== null ? Number(p.disasterPrice) : null,
  };
}

export async function listPositions(userId: number, limit = 50): Promise<PositionRow[]> {
  const db = getDb();
  const rows = await db.select().from(positions).where(eq(positions.userId,
userId)).orderBy(desc(positions.id)).limit(Math.min(limit, 200));
  return rows.map(toPositionRow);
}

export interface PnlSummary {
  realizedTotal: number;
  unrealizedTotal: number;
  unmarkedPositions: number; // open positions with no live mark
  openCount: number;
  closedCount: number;
  winCount: number;
  lossCount: number;
  note: string;
}

export async function getPnlSummary(userId: number): Promise<PnlSummary> {
  const rows = await listPositions(userId, 200);
  const closed = rows.filter((r) => r.status === "CLOSED");
  const open = rows.filter((r) => r.status === "OPEN");
  const realizedTotal = +closed.reduce((a, r) => a + (r.realizedPnl ?? 0), 0).toFixed(2);
  const unrealizedTotal = +open.reduce((a, r) => a + (r.unrealizedPnl ?? 0), 0).toFixed(2);
  const unmarkedPositions = open.filter((r) => r.unrealizedPnl === null).length;
  return {
    realizedTotal,
    unrealizedTotal,
    unmarkedPositions,
    openCount: open.length,
    closedCount: closed.length,
    winCount: closed.filter((r) => (r.realizedPnl ?? 0) > 0).length,
    lossCount: closed.filter((r) => (r.realizedPnl ?? 0) <= 0).length,
    note: unmarkedPositions > 0
      ? `${unmarkedPositions} open position(s) have no live mark — unrealized P&L excludes them (track the
symbol in the data feed).`
      : "All open positions marked to live feed prices.",
  };
}

/* ---------- trailing-stop support ---------- */

/** Every OPEN position carrying a governed stop, across all users — the trailing module's universe. */
export async function listManagedPositions(): Promise<Array<typeof positions.$inferSelect>> {
  const db = getDb();
  return db.select().from(positions).where(and(eq(positions.status, "OPEN"), sql`${positions.stopPrice} IS
```

```
NOT NULL`));
}

/** Book a trailing-module exit: close the row at the actual fill with the fire reason. */
export async function closePositionFromFire(positionId: number, fillPrice: number, reason: "FLAT_STOP" |
"TRAIL" | "SAFETY_FLATTEN" | "MANUAL_FLATTEN" | "DISASTER_STOP"): Promise<void> {
  const db = getDb();
  const [p] = await db.select().from(positions).where(eq(positions.id, positionId)).limit(1);
  if (!p || p.status !== "OPEN") return;
  const realized = (fillPrice - Number(p.avgEntry)) * p.quantity;
  await db.update(positions).set({
    status: "CLOSED",
    closedAt: new Date(),
    exitPrice: String(fillPrice),
    realizedPnl: realized.toFixed(2),
    closedBy: reason,
  }).where(eq(positions.id, positionId));
}

/** Persist trailing-state progress (highest / trail / armed) after each evaluation. */
export async function updateTrailState(positionId: number, fields: { highestPrice?: string; trailPrice?:
string; trailArmed?: boolean }): Promise<void> {
  const db = getDb();
  await db.update(positions).set(fields).where(eq(positions.id, positionId));
}

/** Book a T1/T2 scale-out: split qty off the open row into a CLOSED row, advance the trailing floor
stage. */
export async function bookScaleOut(positionId: number, qty: number, fillPrice: number, stage: 1 | 2):
Promise<void> {
  const db = getDb();
  const [p] = await db.select().from(positions).where(eq(positions.id, positionId)).limit(1);
  if (!p || p.status !== "OPEN") return;
  const closingQty = Math.min(qty, p.quantity);
  if (closingQty <= 0) return;
  const realized = (fillPrice - Number(p.avgEntry)) * closingQty;
  if (closingQty >= p.quantity) {
    await db.update(positions).set({
      status: "CLOSED", closedAt: new Date(), exitPrice: String(fillPrice),
      realizedPnl: realized.toFixed(2), closedBy: stage === 1 ? "SCALE_T1" : "SCALE_T2", scaleStage:
stage,
    }).where(eq(positions.id, p.id));
    return;
  }
  await db.update(positions).set({ quantity: p.quantity - closingQty, scaleStage: stage
}).where(eq(positions.id, p.id));
  await db.insert(positions).values({
    userId: p.userId, symbol: p.symbol, quantity: closingQty, avgEntry: p.avgEntry,
    sourceTicketId: p.sourceTicketId, status: "CLOSED", closedAt: new Date(),
    exitPrice: String(fillPrice), realizedPnl: realized.toFixed(2),
    closedBy: stage === 1 ? "SCALE_T1" : "SCALE_T2",
  });
}

/* ---------- stop mode (per user) ---------- */

export type StopMode = "CLASSIC" | "IMMEDIATE_TRAIL";

export async function getStopMode(userId: number): Promise<StopMode> {
  try {
    const db = getDb();
    const [u] = await db.select({ stopMode: users.stopMode }).from(users).where(eq(users.id,
userId)).limit(1);
    return u?.stopMode === "IMMEDIATE_TRAIL" ? "IMMEDIATE_TRAIL" : "CLASSIC";
  } catch {
    return "CLASSIC"; // fail to the conservative mode
  }
}
```

```
export async function setStopMode(userId: number, mode: StopMode): Promise<{ mode: StopMode }> {
  const db = getDb();
  await db.update(users).set({ stopMode: mode }).where(eq(users.id, userId));
  return { mode };
}

/* ---------- manual overrides ---------- */

async function ownedOpen(userId: number, positionId: number): Promise<typeof positions.$inferSelect |
null> {
  const db = getDb();
  const [p] = await db.select().from(positions)
    .where(and(eq(positions.id, positionId), eq(positions.userId, userId), eq(positions.status,
"OPEN"))).limit(1);
  return p ?? null;
}

async function overrideAlert(userId: number, symbol: string, title: string, body: string): Promise<void> {
  const db = getDb();
  await db.insert(alerts).values({ userId, type: "POSITION_UPDATE", priority: "HIGH", title, body, symbol,
state: "ACTIVE" }).catch(() => undefined);
}

/** Tighten the flat stop upward (toward price). Never lowers it — loosening requires manual control. */
export async function tightenStop(userId: number, positionId: number, newStop: number): Promise<{ ok:
boolean; message: string }> {
  const p = await ownedOpen(userId, positionId);
  if (!p) return { ok: false, message: "No open position with that id." };
  if (!(newStop > 0)) return { ok: false, message: "Stop must be positive." };
  const mark = marketDataService.get(p.symbol)?.indicators?.last ?? null;
  if (mark !== null && newStop >= mark) return { ok: false, message: `Stop ${newStop} is at/above the
current mark ${mark} — use Flatten if you want out now.` };
  const cur = p.stopPrice !== null ? Number(p.stopPrice) : null;
  if (cur !== null && newStop <= cur) return { ok: false, message: `New stop must be ABOVE the current
${cur} — tightening only. Take manual control to loosen.` };
  const db = getDb();
  await db.update(positions).set({ stopPrice: newStop.toFixed(4) }).where(eq(positions.id, p.id));
  await overrideAlert(userId, p.symbol, `Stop tightened — ${p.symbol}`, `Flat stop moved ${cur !== null ?
`${cur} → ` : "to "}${newStop} by the user. Note: changing the stop changes 1R, which moves the trail's
arm threshold (CLASSIC mode).`);
  return { ok: true, message: `Stop tightened to ${newStop}.` };
}

/** Tighten the trail upward. Setting a trail on an unarmed position arms it at that level. */
export async function tightenTrail(userId: number, positionId: number, newTrail: number): Promise<{ ok:
boolean; message: string }> {
  const p = await ownedOpen(userId, positionId);
  if (!p) return { ok: false, message: "No open position with that id." };
  if (!(newTrail > 0)) return { ok: false, message: "Trail must be positive." };
  const mark = marketDataService.get(p.symbol)?.indicators?.last ?? null;
  if (mark !== null && newTrail >= mark) return { ok: false, message: `Trail ${newTrail} is at/above the
current mark ${mark} — use Flatten if you want out now.` };
  const cur = p.trailPrice !== null ? Number(p.trailPrice) : null;
  if (cur !== null && newTrail <= cur) return { ok: false, message: `New trail must be ABOVE the current
${cur} — the ratchet never moves down. Take manual control to loosen.` };
  const db = getDb();
  await db.update(positions).set({ trailPrice: newTrail.toFixed(4), trailArmed: true
}).where(eq(positions.id, p.id));
  await overrideAlert(userId, p.symbol, `Trail tightened — ${p.symbol}`, `Trail ${cur !== null ? `${cur} →
` : "set at "}${newTrail} by the user${p.trailArmed !== true ? " (trail now armed at your level)" : ""}.
The module ratchets upward from here.`);
  return { ok: true, message: `Trail set to ${newTrail}.` };
}

/** Take or return manual control of a position's exit. MANUAL = the module stands down entirely. */
export async function setPositionOverride(userId: number, positionId: number, manual: boolean): Promise<{
ok: boolean; message: string }> {
```

```
  const p = await ownedOpen(userId, positionId);
  if (!p) return { ok: false, message: "No open position with that id." };
  const db = getDb();
  await db.update(positions).set({ overrideMode: manual ? "MANUAL" : null }).where(eq(positions.id,
p.id));
  await overrideAlert(userId, p.symbol,
    manual ? `MANUAL CONTROL — ${p.symbol}` : `Module control resumed — ${p.symbol}`,
    manual
      ? `Position #${p.id} (${p.symbol} ${p.quantity} sh) exit is now HUMAN-MANAGED — no trail updates, no
automatic fires, no scale-outs, no safety flatten. Any resting disaster stop stays in place as insurance.
Return control to resume automatic protection.`
      : `Position #${p.id} (${p.symbol}) returned to the trailing module — automatic protection resumes
next cycle.`);
  return { ok: true, message: manual ? "You now own this exit — the module has stood down." : "Automatic
protection resumed." };
}

/* ---------- live activity stream ---------- */

export interface ActivityEvent {
  id: number;
  kind: string; // alert type
  priority: string;
  title: string;
  body: string | null;
  symbol: string | null;
  at: Date;
}

/** The live trade stream: execution-relevant alerts, newest first. */
export async function streamActivity(userId: number, limit = 30): Promise<ActivityEvent[]> {
  const db = getDb();
  const rows = await db.select().from(alerts)
    .where(and(eq(alerts.userId, userId), sql`${alerts.type} IN
('EXECUTION','PROTECTION','POSITION_UPDATE','CONFIRMATION_REQUEST','RISK')`))
    .orderBy(desc(alerts.id))
    .limit(Math.min(limit, 100));
  return rows.map((a) => ({
    id: Number(a.id),
    kind: a.type,
    priority: a.priority,
    title: a.title,
    body: a.body ?? null,
    symbol: a.symbol ?? null,
    at: a.createdAt,
  }));
}
```

Appendix C — db/schema.ts edits

Three edits: the per-user stop-mode preference, the second target on order tickets, and the full trailing column set on positions (this supersedes the v2.3 column list — apply all of them).

db/schema.ts — users table: add stopMode

```
  stopMode: varchar("stopMode", { length: 16 }).notNull().default("CLASSIC"), // CLASSIC (flat stop → 1R
arm → ATR trail) | IMMEDIATE_TRAIL (0.5% trail from entry + moving hard bottom)
```

db/schema.ts — order_tickets table: add target2

```
  target2: decimal("target2", { precision: 12, scale: 2 }), // second scale-out target (engine proposals)
```

db/schema.ts — positions table: trailing & exit columns

```
/* ---------- Portfolio: positions opened from fills, for P&L ---------- */

export const positions = mysqlTable("positions", {
  id: serial("id").primaryKey(),
  userId: bigint("userId", { mode: "number", unsigned: true }).notNull(),
  symbol: varchar("symbol", { length: 16 }).notNull(),
  quantity: int("quantity").notNull(),
  avgEntry: decimal("avgEntry", { precision: 12, scale: 4 }).notNull(),
  sourceTicketId: varchar("sourceTicketId", { length: 64 }),
  status: mysqlEnum("status", ["OPEN", "CLOSED"]).notNull().default("OPEN"),
  openedAt: timestamp("openedAt").defaultNow().notNull(),
  closedAt: timestamp("closedAt"),
  exitPrice: decimal("exitPrice", { precision: 12, scale: 4 }),
  realizedPnl: decimal("realizedPnl", { precision: 12, scale: 2 }),
  /* --- trailing-stop state (api/engine/trailing.ts) --- */
  stopPrice: decimal("stopPrice", { precision: 12, scale: 4 }), // original flat stop — governs until the
trail arms
  t1Price: decimal("t1Price", { precision: 12, scale: 4 }), // first target — floor reference after scale-
outs
  highestPrice: decimal("highestPrice", { precision: 12, scale: 4 }), // highest price since entry
  trailPrice: decimal("trailPrice", { precision: 12, scale: 4 }), // current trail (never moves down)
  trailArmed: boolean("trailArmed").notNull().default(false), // armed once price reaches entry + 1R
(CLASSIC) or at entry (IMMEDIATE_TRAIL)
  scaleStage: int("scaleStage").notNull().default(0), // 0 = full size, 1 = after T1 scale-out, 2 = after
T2
  closedBy: varchar("closedBy", { length: 24 }), // FLAT_STOP | TRAIL | SAFETY_FLATTEN | MANUAL_FLATTEN |
DISASTER_STOP | null (ticket close)
  t2Price: decimal("t2Price", { precision: 12, scale: 4 }), // second target — T2 scale-out trigger
  overrideMode: varchar("overrideMode", { length: 16 }), // null = module-managed | MANUAL = human owns
the exit
  disasterOrderId: varchar("disasterOrderId", { length: 64 }), // resting broker-side catastrophe stop
(null on paper)
  disasterPrice: decimal("disasterPrice", { precision: 12, scale: 4 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});
export type Position = typeof positions.$inferSelect;
```

Appendix D — api/lib/ensure-schema.ts additions

Guarded ALTER statements, appended to the existing boot-time migration block. Each is idempotent (duplicate-column errors are swallowed), so first boot upgrades the live database and reboots are no-ops.

api/lib/ensure-schema.ts — append to the ALTER list

```
// Trailing-stop state on positions (trailing module)
`ALTER TABLE positions ADD COLUMN stopPrice DECIMAL(12,4) NULL AFTER realizedPnl`,
`ALTER TABLE positions ADD COLUMN t1Price DECIMAL(12,4) NULL AFTER stopPrice`,
`ALTER TABLE positions ADD COLUMN highestPrice DECIMAL(12,4) NULL AFTER t1Price`,
`ALTER TABLE positions ADD COLUMN trailPrice DECIMAL(12,4) NULL AFTER highestPrice`,
`ALTER TABLE positions ADD COLUMN trailArmed TINYINT(1) NOT NULL DEFAULT 0 AFTER trailPrice`,
`ALTER TABLE positions ADD COLUMN scaleStage INT NOT NULL DEFAULT 0 AFTER trailArmed`,
`ALTER TABLE positions ADD COLUMN closedBy VARCHAR(24) NULL AFTER scaleStage`,
// Final autonomous update: stop modes, T2 scale-outs, overrides, disaster stop
`ALTER TABLE users ADD COLUMN stopMode VARCHAR(16) NOT NULL DEFAULT 'CLASSIC' AFTER autoExecute`,
`ALTER TABLE order_tickets ADD COLUMN target2 DECIMAL(12,2) NULL AFTER target`,
`ALTER TABLE positions ADD COLUMN t2Price DECIMAL(12,4) NULL AFTER closedBy`,
`ALTER TABLE positions ADD COLUMN overrideMode VARCHAR(16) NULL AFTER t2Price`,
`ALTER TABLE positions ADD COLUMN disasterOrderId VARCHAR(64) NULL AFTER overrideMode`,
`ALTER TABLE positions ADD COLUMN disasterPrice DECIMAL(12,4) NULL AFTER disasterOrderId`,
```

Appendix E — broker layer: open-order listing

The optional capability that powers cancel-and-govern and disaster-stop polling. Adapters that cannot list open orders simply omit the method — the module degrades to MEDIUM alerts instead of failing.

api/brokers/types.ts — add the BrokerOpenOrder interface

```
export interface BrokerOpenOrder {
  brokerOrderId: string;
  symbol: string | null; // ticker when the broker reports one (null = match by other means)
  side: "BUY" | "SELL" | null;
  orderType: string; // broker-native string, uppercased by the adapter
  quantity: number | null;
  limitPrice: number | null;
  stopPrice: number | null;
  clientOrderId: string | null;
  status: string;
```

api/brokers/types.ts — add the optional method to BrokerAdapter

```
  /** Live (unfilled, uncancelled) orders — optional; adapters without an open-orders endpoint omit it. */
  getOpenOrders?(accountId: string): Promise<BrokerOpenOrder[]>;
```

api/brokers/ibkr.ts — implement getOpenOrders

```
  /** Live orders from the Client Portal — used by the trailing module to reconcile broker-side stops. */
  async getOpenOrders(accountId: string): Promise<BrokerOpenOrder[]> {
    const out = await this.req<{ orders?: Array<Record<string, unknown>> }>("GET",
`/iserver/account/${accountId}/orders`);
    const rows = out?.orders ?? [];
    return rows
      .filter((o) => !/fill|cancel/i.test(String(o.status ?? "")))
      .map((o) => ({
        brokerOrderId: String(o.orderId ?? o.order_id ?? ""),
        symbol: o.ticker !== undefined ? String(o.ticker) : null,
        side: o.side !== undefined ? (String(o.side).toUpperCase().startsWith("S") ? "SELL" : "BUY") :
null,
        orderType: String(o.orderType ?? o.order_type ?? "").toUpperCase(),
        quantity: o.totalSize !== undefined ? Number(o.totalSize) : o.quantity !== undefined ?
Number(o.quantity) : null,
        limitPrice: o.price !== undefined && o.price !== null ? Number(o.price) : null,
        stopPrice: o.auxPrice !== undefined && o.auxPrice !== null ? Number(o.auxPrice) : null,
        clientOrderId: o.cOID !== undefined && o.cOID !== null ? String(o.cOID) : null,
        status: String(o.status ?? ""),
      }));
  }
```

Appendix F — target2 plumbing (tickets & monitor)

Two one-line edits so the second target price flows from the scanner's proposal into the ticket row, and from there into `recordFill` (Appendix B) where it seeds `t2Price`.

api/queries/tickets.ts — carry target2 on insert

```
// ProposeInput gains:
target2?: number;

// and the insert carries it:
  target2: input.target2 !== undefined ? String(input.target2) : null,
```

api/engine/monitor.ts — proposeTicket call gains target2

```
          entry: proposal.entry,
          stop: proposal.stop,
          target: proposal.targets[0]?.price,
          target2: proposal.targets[1]?.price,
          origin: "AUTONOMOUS",
        });
        ticket = result.ticket;
```

Appendix G — api/engine-router.ts additions

The new tRPC surface: trailing status, stop-mode get/set, override mutations, manual flatten, and the activity stream. Add inside the existing engine router.

api/engine-router.ts — add these procedures

```
  /** Trailing-stop module status (running, managed positions, fires today). */
  trailing: authedQuery.query(() => trailingStatus()),

  /** Per-user stop mode: CLASSIC (flat stop → 1R arm → ATR trail) or IMMEDIATE_TRAIL (0.5% trail from
entry + moving hard bottom). */
  getStopMode: authedQuery.query(async ({ ctx }) => ({ mode: await getStopMode(ctx.user.id) })),
  setStopMode: authedQuery
    .input(z.object({ mode: z.enum(["CLASSIC", "IMMEDIATE_TRAIL"]) }))
    .mutation(async ({ ctx, input }) => setStopMode(ctx.user.id, input.mode)),

  /** Manual overrides — tighten only (ratchet-respecting); loosening requires manual control. */
  tightenStop: authedQuery
    .input(z.object({ positionId: z.number().int().positive(), stopPrice: z.number().positive() }))
    .mutation(({ ctx, input }) => tightenStop(ctx.user.id, input.positionId, input.stopPrice)),
  tightenTrail: authedQuery
    .input(z.object({ positionId: z.number().int().positive(), trailPrice: z.number().positive() }))
    .mutation(({ ctx, input }) => tightenTrail(ctx.user.id, input.positionId, input.trailPrice)),
  setPositionOverride: authedQuery
    .input(z.object({ positionId: z.number().int().positive(), manual: z.boolean() }))
    .mutation(({ ctx, input }) => setPositionOverride(ctx.user.id, input.positionId, input.manual)),
  /** Flatten an open position now — market sell through the same audited fire path. */
  flattenPosition: authedQuery
    .input(z.object({ positionId: z.number().int().positive() }))
    .mutation(({ ctx, input }) => requestFlatten(ctx.user.id, input.positionId)),

  /** Live trade stream — execution-relevant events, newest first. */
  activity: authedQuery.query(({ ctx }) => streamActivity(ctx.user.id, 40)),
```

Appendix H — boot wiring & system-check probe

Two edits to start the loop in production, and the 12th health probe that FAILs if the loop dies during market hours.

api/boot.ts — import (top of file)

```
import { ensureTrailingLoop } from "./engine/trailing";
```

api/boot.ts — inside the production listen block, after ensureSystemCheckLoop()

```
  ensureTrailingLoop(); // 5s protective-exit cycle during market hours — flat stop, 2.5×ATR trail, safety flatten
```

api/systemcheck/checks.ts — add the checkTrailing probe

```
/* ---------- 12. trailing-stop module ---------- */

const checkTrailing: CheckFn = async () => {
  const id = "trailing";
  const name = "Trailing-stop module";
  try {
    const s = trailingStatus();
    if (!s.running) {
      return fail(id, name, "Trailing loop is NOT running — no protective exits are being managed",
        "Boot the server with NODE_ENV=production node dist/boot.js — ensureTrailingLoop() only starts in the production block.");
    }
    if (!s.marketHours) {
      return ok(id, name, `Trailing loop running (sleeps off-hours by design — positions rest on their flat stops) · ${s.firedToday} fire(s) today`);
    }
    const stale = s.lastTickAt === null || Date.now() - s.lastTickAt > 30_000;
    if (stale) {
      return fail(id, name, "Trailing loop started but has not ticked in >30s during market hours — protective exits may be stalled",
        "Check server logs for tick errors (DB connectivity or market-data service). Restart the production process if ticks do not resume.");
    }
    return ok(id, name, `Managing ${s.managed} open position(s) · ${s.inFlight} exit order(s) in flight · ${s.firedToday} fire(s) today`);
  } catch (e) {
    return fail(id, name, `Trailing module error: ${(e as Error).message}`,
      "Import or evaluation failure in api/engine/trailing.ts — check server logs.");
  }
};
```

Appendix I — src/components/OrdersPanel.tsx (full replacement)

The Orders & P&L panel with the stop-mode toggle, trailing-stops status card, Protection column (trail / floor / disaster / manual badges), and the Actions column (TIGHTEN, MANUAL/AUTO, FLATTEN). Replace the v2.2 file.

src/components/OrdersPanel.tsx — replace the v2.2 file

```
import { trpc } from '@/providers/trpc';
import { Badge } from '@/pages/app/owner/shared';
import { Zap, ZapOff, RefreshCw, TrendingUp, TrendingDown } from 'lucide-react';

interface OrderRow {
  ticketId: string; symbol: string; side: string; quantity: number; orderType: string;
  state: string; autoExecuted: boolean; entry: number | null; stop: number | null;
  target: number | null; filledQuantity: number | null; averageFillPrice: number | null;
  createdAt: string | Date; lastMessage: string | null;
}
interface PositionRow {
  id: number; symbol: string; quantity: number; avgEntry: number; status: string;
  mark: number | null; unrealizedPnl: number | null; realizedPnl: number | null;
  stopPrice: number | null; highestPrice: number | null; trailPrice: number | null;
  trailArmed: boolean; scaleStage: number; closedBy: string | null;
  t2Price: number | null; overrideMode: string | null; disasterPrice: number | null;
  openedAt: string | Date;
}

function stateTone(s: string): 'teal' | 'amber' | 'red' | 'slate' | 'sky' {
  if (s === 'FILLED') return 'teal';
  if (s === 'WORKING' || s === 'SUBMITTING' || s === 'CONFIRMED') return 'sky';
  if (s === 'FAILED' || s === 'REJECTED') return 'red';
  if (s === 'EXPIRED') return 'amber';
  return 'slate';
}

export default function OrdersPanel() {
  const utils = trpc.useUtils();
  const { data: pnl, refetch } = trpc.engine.pnl.useQuery(undefined, { refetchInterval: 15000 });
  const { data: positions } = trpc.engine.positions.useQuery(undefined, { refetchInterval: 15000 });
  const { data: orders } = trpc.engine.orders.useQuery(undefined, { refetchInterval: 15000 });
  const { data: autoExec } = trpc.engine.getAutoExecute.useQuery(undefined);
  const { data: trailing } = trpc.engine.trailing.useQuery(undefined, { refetchInterval: 15000 });
  const { data: stopMode } = trpc.engine.getStopMode.useQuery(undefined);
  const setMode = trpc.engine.setStopMode.useMutation({
    onSuccess: () => utils.engine.getStopMode.invalidate(),
  });
  const tightenStopMut = trpc.engine.tightenStop.useMutation({ onSuccess: () => {
utils.engine.positions.invalidate(); utils.engine.activity.invalidate(); } });
  const tightenTrailMut = trpc.engine.tightenTrail.useMutation({ onSuccess: () => {
utils.engine.positions.invalidate(); utils.engine.activity.invalidate(); } });
  const overrideMut = trpc.engine.setPositionOverride.useMutation({ onSuccess: () => {
utils.engine.positions.invalidate(); utils.engine.activity.invalidate(); } });
  const flattenMut = trpc.engine.flattenPosition.useMutation({ onSuccess: () => {
utils.engine.positions.invalidate(); utils.engine.activity.invalidate(); utils.engine.pnl.invalidate(); }
});
  const immediate = stopMode?.mode === 'IMMEDIATE_TRAIL';
  const setAuto = trpc.engine.setAutoExecute.useMutation({
    onSuccess: () => utils.engine.getAutoExecute.invalidate(),
  });

  const autoOn = autoExec?.enabled === true;
  const openPositions = (positions ?? []).filter((p: PositionRow) => p.status === 'OPEN');
```

```
  return (
    <section className="glass rounded-2xl p-6">
      <div className="mb-4 flex flex-wrap items-center justify-between gap-3">
        <div>
          <h2 className="font-display text-lg font-bold text-slate-900">Orders &amp; P&amp;L</h2>
          <p className="text-xs text-slate-500">Order history, open positions, and the auto-execute switch
for autonomous proposals.</p>
        </div>
        <div className="flex items-center gap-3">
          <button onClick={() => refetch()} className="inline-flex items-center gap-1.5 text-xs font-
semibold text-slate-500 hover:text-slate-700">
            <RefreshCw className="h-3.5 w-3.5" /> Refresh
          </button>
          <button
            onClick={() => setMode.mutate({ mode: immediate ? 'CLASSIC' : 'IMMEDIATE_TRAIL' })}
            disabled={setMode.isPending}
            title={immediate ? 'Trail live from entry at 0.5%, tightening on gains, with a moving hard
bottom — no flat stop' : 'Flat stop until +1R, then a 2.5×ATR trail'}
            className={`inline-flex items-center gap-2 rounded-xl px-3 py-2 text-xs font-semibold
transition border ${
              immediate
                ? 'bg-teal-500/15 text-teal-700 border-teal-500/30 hover:bg-teal-500/25'
                : 'bg-slate-900/[0.05] text-slate-600 border-slate-900/10 hover:bg-slate-900/10'
            }`}
          >
            {immediate ? 'Trail: IMMEDIATE' : 'Trail: CLASSIC'}
          </button>
          <button
            onClick={() => setAuto.mutate({ enabled: !autoOn })}
            disabled={setAuto.isPending}
            className={`inline-flex items-center gap-2 rounded-xl px-4 py-2 text-sm font-semibold
transition ${
              autoOn
                ? 'bg-amber-500/15 text-amber-700 border border-amber-500/30 hover:bg-amber-500/25'
                : 'bg-slate-900/[0.05] text-slate-600 border border-slate-900/10 hover:bg-slate-900/10'
            }`}
          >
            {autoOn ? <Zap className="h-4 w-4" /> : <ZapOff className="h-4 w-4" />}
            Auto-execute {autoOn ? 'ON' : 'OFF'}
          </button>
        </div>
      </div>

      {autoOn && (
        <div className="mb-4 rounded-xl border border-amber-500/30 bg-amber-500/10 px-4 py-2.5 text-xs
text-amber-700">
          <b>Auto-execute is ON.</b> Autonomous engine proposals will buy <b>without</b> asking for your
CONFIRM string — every fill still creates a ticket, an alert, and an order-history entry marked AUTO.
Natural-language (chat) trades still always require confirmation. Turn this off anytime to return to
confirm-first mode.
        </div>
      )}

      {immediate && (
        <div className="mb-4 rounded-xl border border-teal-500/30 bg-teal-500/10 px-4 py-2.5 text-xs text-
teal-700">
          <b>Immediate-trail mode.</b> No flat stop-loss is used anywhere — a software trail is live from
entry at 0.5% below price, tightening as the trade gains (0.50 → 0.45 → 0.40 → 0.32 → 0.27 → 0.22%), with
a hard bottom that moves from entry −0.5% to breakeven at +1% and locks 40% of peak gains beyond +2%. Both
ratchets only move up.
        </div>
      )}

      {/* P&L summary */}
      <div className="mb-5 grid gap-3 sm:grid-cols-4">
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Realized P&amp;L</p>
```

```
          <p className={`mt-1 text-xl font-bold ${(pnl?.realizedTotal ?? 0) >= 0 ? 'text-teal-600' :
'text-red-600'}`}>
            {(pnl?.realizedTotal ?? 0) >= 0 ? '+' : ''}${(pnl?.realizedTotal ?? 0).toLocaleString()}
          </p>
          <p className="text-[11px] text-slate-500">{pnl?.closedCount ?? 0} closed · {pnl?.winCount ?? 0}W
/ {pnl?.lossCount ?? 0}L</p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Unrealized P&amp;amp;L</p>
          <p className={`mt-1 text-xl font-bold ${(pnl?.unrealizedTotal ?? 0) >= 0 ? 'text-teal-600' :
'text-red-600'}`}>
            {(pnl?.unrealizedTotal ?? 0) >= 0 ? '+' : ''}${(pnl?.unrealizedTotal ?? 0).toLocaleString()}
          </p>
          <p className="text-[11px] text-slate-500">{(pnl?.unmarkedPositions ?? 0) > 0 ?
`${pnl?.unmarkedPositions} unmarked` : 'marked to feed'}</p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Open positions</p>
          <p className="mt-1 text-xl font-bold text-slate-900">{pnl?.openCount ?? 0}</p>
          <p className="text-[11px] text-slate-500">{openPositions.map((p: PositionRow) =>
p.symbol).slice(0, 4).join(', ') || '—'}</p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Trailing stops</p>
          <p className={`mt-1 text-xl font-bold ${trailing?.running ? 'text-teal-600' : 'text-red-600'}`}>
            {trailing?.running ? (trailing.marketHours ? 'LIVE' : 'ARMED') : 'DOWN'}
          </p>
          <p className="text-[11px] text-slate-500">
            {trailing?.running
              ? `${trailing.managed} protected · ${trailing.firedToday} fired today${trailing.marketHours
? '' : ' · sleeps off-hours'}`
              : 'not running — check System Check'}
          </p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Execution mode</p>
          <p className={`mt-1 text-xl font-bold ${autoOn ? 'text-amber-600' : 'text-slate-900'}`}>{autoOn
? 'AUTO' : 'CONFIRM'}</p>
          <p className="text-[11px] text-slate-500">{autoOn ? 'no confirmation required' : 'CONFIRM string
required'}</p>
        </div>
      </div>

      {/* open positions */}
      {openPositions.length > 0 && (
        <div className="mb-5">
          <h3 className="mb-2 text-sm font-semibold text-slate-800">Open positions</h3>
          <div className="overflow-x-auto">
            <table className="w-full text-sm">
              <thead>
                <tr className="border-b border-slate-900/5 text-left text-[11px] uppercase tracking-wide
text-slate-500">
                  <th className="px-3 py-2">Symbol</th><th className="px-3 py-2">Qty</th><th
className="px-3 py-2">Avg entry</th>
                  <th className="px-3 py-2">Mark</th><th className="px-3 py-2">Protection</th><th
className="px-3 py-2">Unrealized</th><th className="px-3 py-2">Actions</th>
                </tr>
              </thead>
              <tbody>
                {openPositions.map((p: PositionRow) => (
                  <tr key={p.id} className="border-b border-slate-900/[0.04] last:border-0">
                    <td className="px-3 py-2 font-semibold text-slate-800">{p.symbol}</td>
                    <td className="px-3 py-2 text-slate-600">{p.quantity}</td>
                    <td className="px-3 py-2 text-slate-600">${p.avgEntry.toFixed(2)}</td>
                    <td className="px-3 py-2 text-slate-600">{p.mark !== null ? `$${p.mark.toFixed(2)}` :
'no mark'}</td>
                    <td className="px-3 py-2">
                      {p.overrideMode === 'MANUAL' ? (
```

```
                        <span className="rounded-full bg-red-100 px-1.5 py-0.5 text-[9px] font-bold text-
red-700">MANUAL — module OFF</span>
                      ) : p.trailArmed && p.trailPrice !== null ? (
                        <span className="inline-flex items-center gap-1">
                          <span className="rounded-full bg-amber-100 px-1.5 py-0.5 text-[9px] font-bold
text-amber-700">TRAIL</span>
                          <span className="text-slate-700">${p.trailPrice.toFixed(2)}</span>
                          {p.scaleStage > 0 && <span className="rounded-full bg-sky-100 px-1.5 py-0.5
text-[9px] font-bold text-sky-700">{p.scaleStage >= 2 ? 'T1 FLOOR' : 'BE FLOOR'}</span>}
                          {p.disasterPrice !== null && <span className="rounded-full bg-slate-100 px-1.5
py-0.5 text-[9px] font-bold text-slate-500" title={`Disaster stop resting at broker @
$${p.disasterPrice.toFixed(2)}`}>+D</span>}
                        </span>
                      ) : p.stopPrice !== null ? (
                        <span className="inline-flex items-center gap-1">
                          <span className="text-slate-500">flat ${p.stopPrice.toFixed(2)}</span>
                          {p.disasterPrice !== null && <span className="rounded-full bg-slate-100 px-1.5
py-0.5 text-[9px] font-bold text-slate-500" title={`Disaster stop resting at broker @
$${p.disasterPrice.toFixed(2)}`}>+D</span>}
                        </span>
                      ) : (
                        <span className="text-slate-400">—</span>
                      )}
                    </td>
                    <td className={`px-3 py-2 font-semibold ${(p.unrealizedPnl ?? 0) >= 0 ? 'text-teal-
600' : 'text-red-600'}`}>
                      {p.unrealizedPnl !== null ? (
                        <span className="inline-flex items-center gap-1">
                          {p.unrealizedPnl >= 0 ? <TrendingUp className="h-3.5 w-3.5" /> : <TrendingDown
className="h-3.5 w-3.5" />}
                          {p.unrealizedPnl >= 0 ? '+' : ''}${p.unrealizedPnl.toFixed(2)}
                        </span>
                      ) : '—'}
                    </td>
                    <td className="px-3 py-2">
                      <span className="inline-flex items-center gap-1">
                        <button
                          onClick={() => {
                            const v = window.prompt(`Tighten protection for ${p.symbol} (must be ABOVE
current level, below mark). Enter new ${p.trailArmed ? 'trail' : 'stop'} price:`);
                            if (v === null) return;
                            const n = Number(v);
                            if (!(n > 0)) { window.alert('Invalid price.'); return; }
                            if (p.trailArmed) tightenTrailMut.mutate({ positionId: p.id, trailPrice: n });
                            else tightenStopMut.mutate({ positionId: p.id, stopPrice: n });
                          }}
                          className="rounded-md bg-slate-900/[0.05] px-1.5 py-0.5 text-[9px] font-bold
text-slate-600 hover:bg-slate-900/10"
                        >TIGHTEN</button>
                        <button
                          onClick={() => overrideMut.mutate({ positionId: p.id, manual: p.overrideMode !==
'MANUAL' })}
                          className={`rounded-md px-1.5 py-0.5 text-[9px] font-bold ${p.overrideMode ===
'MANUAL' ? 'bg-teal-100 text-teal-700 hover:bg-teal-200' : 'bg-amber-100 text-amber-700 hover:bg-amber-
200'}`}
                          title={p.overrideMode === 'MANUAL' ? 'Return to automatic protection' : 'Take
manual control — module stands down (only way to loosen a stop)'}
                        >{p.overrideMode === 'MANUAL' ? 'AUTO' : 'MANUAL'}</button>
                        <button
                          onClick={() => { if (window.confirm(`Flatten ${p.symbol} — market-sell all
${p.quantity} shares now?`)) flattenMut.mutate({ positionId: p.id }); }}
                          className="rounded-md bg-red-100 px-1.5 py-0.5 text-[9px] font-bold text-red-700
hover:bg-red-200"
                        >FLATTEN</button>
                      </span>
                    </td>
                  </tr>
                ))}
```

```
              </tbody>
            </table>
          </div>
        </div>
      )}

      {/* order history */}
      <h3 className="mb-2 text-sm font-semibold text-slate-800">Order history</h3>
      {!orders || orders.length === 0 ? (
        <p className="py-6 text-center text-sm text-slate-500">No orders yet — confirmed and auto-executed
tickets will appear here.</p>
      ) : (
        <div className="overflow-x-auto">
          <table className="w-full text-sm">
            <thead>
              <tr className="border-b border-slate-900/5 text-left text-[11px] uppercase tracking-wide
text-slate-500">
                <th className="px-3 py-2">Ticket</th><th className="px-3 py-2">Symbol</th><th
className="px-3 py-2">Side</th>
                <th className="px-3 py-2">Qty</th><th className="px-3 py-2">State</th><th className="px-3
py-2">Fill</th>
                <th className="px-3 py-2">Stop / Target</th><th className="px-3 py-2">When</th>
              </tr>
            </thead>
            <tbody>
              {(orders as OrderRow[]).map((o) => (
                <tr key={o.ticketId} className="border-b border-slate-900/[0.04] last:border-0">
                  <td className="px-3 py-2 font-mono text-xs text-slate-600">
                    {o.ticketId}
                    {o.autoExecuted && <Badge tone="amber">AUTO</Badge>}
                  </td>
                  <td className="px-3 py-2 font-semibold text-slate-800">{o.symbol}</td>
                  <td className="px-3 py-2 text-slate-600">{o.side}</td>
                  <td className="px-3 py-2 text-slate-600">{o.quantity}</td>
                  <td className="px-3 py-2"><Badge tone={stateTone(o.state)}>{o.state}</Badge></td>
                  <td className="px-3 py-2 text-slate-600">
                    {o.averageFillPrice !== null ? `${o.filledQuantity ?? o.quantity} @
$${o.averageFillPrice.toFixed(2)}` : '—'}
                  </td>
                  <td className="px-3 py-2 text-xs text-slate-500">
                    {o.stop !== null ? `$${o.stop.toFixed(2)}` : '—'} / {o.target !== null ?
`$${o.target.toFixed(2)}` : '—'}
                  </td>
                  <td className="px-3 py-2 text-xs text-slate-500">{new
Date(o.createdAt).toLocaleString()}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </section>
  );
}
```

Appendix J — src/components/LiveStream.tsx (new file) + mount

The live trade stream: polls `engine.activity` every 5 seconds and renders executions, scale-outs, fires, disaster events, and risk alerts with priority-styled icons. Create the component, then mount it on the Autonomous page directly after `<OrdersPanel />`.

src/components/LiveStream.tsx — create as a new file

```
import { useEffect, useRef } from 'react';
import { Activity, ArrowDownUp, Flame, ShieldAlert, ShieldCheck, Info } from 'lucide-react';
import { trpc } from '@/providers/trpc';

interface ActivityEvent {
  id: number;
  kind: string;
  priority: string;
  title: string;
  body: string | null;
  symbol: string | null;
  at: string | Date;
}

function etTime(d: string | Date): string {
  return new Date(d).toLocaleTimeString('en-US', { timeZone: 'America/New_York', hour12: false });
}

function styleFor(e: ActivityEvent): { icon: JSX.Element; bar: string; text: string } {
  if (e.priority === 'CRITICAL') {
    return { icon: <ShieldAlert className="h-3.5 w-3.5 text-red-500" />, bar: 'border-red-500/60', text:
'text-red-700' };
  }
  if (/scale-out/i.test(e.title)) {
    return { icon: <ArrowDownUp className="h-3.5 w-3.5 text-sky-500" />, bar: 'border-sky-500/60', text:
'text-sky-700' };
  }
  if (/stop|trail|flatten|disaster/i.test(e.title)) {
    return { icon: <Flame className="h-3.5 w-3.5 text-amber-500" />, bar: 'border-amber-500/60', text:
'text-amber-700' };
  }
  if (/filled|executed|closed|confirmed/i.test(e.title)) {
    return { icon: <ShieldCheck className="h-3.5 w-3.5 text-teal-500" />, bar: 'border-teal-500/60', text:
'text-teal-700' };
  }
  return { icon: <Info className="h-3.5 w-3.5 text-slate-400" />, bar: 'border-slate-300', text: 'text-
slate-600' };
}

/**
 * LIVE TRADE STREAM — the "watch the bot buy and sell" feed.
 * Entries, scale-outs, trail fires, safety events and overrides as they
 * happen, newest first, polling every 5 seconds.
 */
export default function LiveStream() {
  const { data: events } = trpc.engine.activity.useQuery(undefined, { refetchInterval: 5000 });
  const topRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    topRef.current?.scrollTo({ top: 0 });
  }, [events?.length]);

  const live = (events ?? []).length > 0;
```

```
  return (
    <section className="glass rounded-2xl p-6">
      <div className="mb-3 flex items-center justify-between">
        <div className="flex items-center gap-2">
          <Activity className="h-4 w-4 text-slate-500" />
          <h2 className="font-display text-lg font-bold text-slate-900">Live trade stream</h2>
          <span className="relative flex h-2 w-2">
            <span className="absolute inline-flex h-full w-full animate-ping rounded-full bg-teal-400
opacity-75"></span>
            <span className="relative inline-flex h-2 w-2 rounded-full bg-teal-500"></span>
          </span>
        </div>
        <p className="text-[11px] text-slate-400">updates every 5s · times ET</p>
      </div>

      <div ref={topRef} className="max-h-72 space-y-1.5 overflow-y-auto pr-1">
        {!live && (
          <p className="py-6 text-center text-xs text-slate-400">
            Nothing yet — entries, scale-outs, trailing-stop fires and overrides appear here the moment
they happen.
          </p>
        )}
        {(events ?? []).map((e: ActivityEvent) => {
          const s = styleFor(e);
          return (
            <div key={e.id} className={`rounded-lg border-l-2 bg-white/60 px-3 py-2 ${s.bar}`}>
              <div className="flex items-center gap-2">
                {s.icon}
                <span className="font-mono text-[10px] text-slate-400">{etTime(e.at)}</span>
                {e.symbol && <span className="rounded bg-slate-900/[0.05] px-1 py-0.5 text-[9px] font-bold
text-slate-600">{e.symbol}</span>}
                <span className={`text-[11px] font-semibold ${s.text}`}>{e.title}</span>
              </div>
              {e.body && <p className="mt-0.5 pl-6 text-[10px] leading-snug text-slate-500">{e.body}</p>}
            </div>
          );
        })}
      </div>
    </section>
  );
}
```

src/pages/app/Autonomous.tsx — import (top of file)

```
import LiveStream from '@/components/LiveStream';
```

src/pages/app/Autonomous.tsx — render immediately after <OrdersPanel />

```
<LiveStream />
```